



```
In [2]: #Modulo de llamadas http para descargar ficheros
!pip install requests

#Libreria del problema TSP: http://elib.zib.de/pub/mp-testdata/tsp/tsplib/tsplib95
!pip install tsplib95
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2026.6.17)
Requirement already satisfied: tsplib95 in /usr/local/lib/python3.12/dist-packages (0.7.1)
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (8.4.2)
Requirement already satisfied: Deprecated~=1.2.9 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (1.2.18)
Requirement already satisfied: networkx~=2.1 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (2.8.8)
Requirement already satisfied: tabulate~=0.8.7 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (0.8.10)
Requirement already satisfied: wrapt<2,>=1.10 in /usr/local/lib/python3.12/dist-packages (from Deprecated~=1.2.9->tsplib95) (1.17.3)
```

```
In [10]: import tsplib95
import random
from math import e
import urllib.request
```

```
In [22]: #DATOS DEL PROBLEMA
file = "/content/swiss42.tsp.tsp"
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
    return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion(lista de nodos)
def distancia_total(solucion, problem):
    distancia_total = 0
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i],solucion[i+1], problem)
    return distancia_total + distancia(solucion[len(solucion)-1],solucion[0], p
```

```
In [19]: import os
```

```
print(os.getcwd())
```

/content

Algoritmo de colonia de hormigas

La función Add_Nodo selecciona al azar un nodo con probabilidad uniforme. Para ser mas eficiente debería seleccionar el próximo nodo siguiendo la probabilidad correspondiente a la ecuación:

$$p^{k_{ij}}(t) = \frac{[\tau_{ij}(t)]^{\alpha} [\sum_{l \in J^k_i} [\tau_{il}(t)]^{\alpha}]^{-\beta}}{\sum_{j \in J^k_i} [\tau_{ij}(t)]^{\alpha} [\sum_{l \in J^k_i} [\tau_{il}(t)]^{\alpha}]^{-\beta}}, \text{ si } j \in J^k_i$$

$$p^{k_{ij}}(t) = 0, \text{ si } j \notin J^k_i$$

```
In [23]: def Add_Nodo(problem, H ,T ) :  
    #Mejora:Establecer una funcion de probabilidad para  
    # añadir un nuevo nodo dependiendo de los nodos mas cercanos y de las ferom  
    Nodos = list(problem.get_nodes())  
    return random.choice( list(set(range(1,len(Nodos))) - set(H) ) )  
  
def Incrementa_Feromona(problem, T, H ) :  
    #Incrementa segun la calidad de la solución. Añadir una cantidad inversament  
    for i in range(len(H)-1):  
        T[H[i]][H[i+1]] += 1000/distancia_total(H, problem)  
    return T  
  
def Evaporar_Feromonas(T ):  
    #Evapora 0.3 el valor de la feromona, sin que baje de 1  
    #Mejora:Podemos elegir diferentes funciones de evaporación dependiendo de la  
    T = [[ max(T[i][j] - 0.3 , 1) for i in range(len(Nodos)) ] for j in range(le  
    return T
```

```
In [24]: def hormigas(problem, N) :  
    #problem = datos del problema  
    #N = Número de agentes(hormigas)  
  
    #Nodos  
    Nodos = list(problem.get_nodes())  
    #Aristas  
    Aristas = list(problem.get_edges())  
  
    #Inicializa las aristas con una cantidad inicial de feromonas:1  
    #Mejora: inicializar con valores diferentes dependiendo diferentes criterios  
    T = [[ 1 for _ in range(len(Nodos)) ] for _ in range(len(Nodos))]  
  
    #Se generan los agentes(hormigas) que serán estructuras de caminos desde 0  
    Hormiga = [[0] for _ in range(N)]  
  
    #Recorre cada agente construyendo la solución
```

```

for h in range(N) :
    #Para cada agente se construye un camino
    for i in range(len(Nodos)-1) :

        #Elige el siguiente nodo
        Nuevo_Nodo = Add_Nodo(problem, Hormiga[h] ,T )
        Hormiga[h].append(Nuevo_Nodo)

        #Incrementa feromonas en esa arista
        T = Incrementa_Feromona(problem, T, Hormiga[h] )
        #print("Feromonas(1)", T)

        #Evapora Feromonas
        T = Evaporar_Feromonas(T)
        #print("Feromonas(2)", T)

        #Seleccionamos el mejor agente
        mejor_solucion = []
        mejor_distancia = 10e100
        for h in range(N) :
            distancia_actual = distancia_total(Hormiga[h], problem)
            if distancia_actual < mejor_distancia:
                mejor_solucion = Hormiga[h]
                mejor_distancia = distancia_actual

        print(mejor_solucion)
        print(mejor_distancia)

hormigas(problem, 1000)

```

[0]

1